

Customer Churn Prediction

ANN Architecture, Performance & Explainability Report

1. Methodology & Data Preparation

This report documents the complete predictive modeling pipeline for customer churn prediction. We started with 7,043 raw customer records from a telecommunications company. After removing 302 duplicates and handling missing values and blank strings, we retained 6,741 clean records. These were split 80/20 into training (5,392) and test (1,349) sets using stratified sampling to preserve the original class distribution.

1.1 Feature Engineering: K-Means Cluster Variable

The K-Means clustering model from Stage 2 (Clustering Analytics) was loaded and used to predict cluster labels for every customer based on their 'tenure' and 'MonthlyCharges' values. This cluster label was appended as an additional (11th) input feature. The rationale is that customer segments identified by unsupervised learning can provide the supervised model with a powerful behavioural grouping signal -- customers in different lifecycle clusters have fundamentally different churn risk profiles.

1.2 Class Imbalance & SMOTE

The target variable (Churn) is imbalanced: 73.4% of customers did not churn vs 26.6% who did. If we trained the model naively, it could achieve ~73% accuracy by predicting 'No Churn' for everyone -- a useless model for business purposes.

To address this, we applied SMOTE (Synthetic Minority Oversampling Technique) ONLY to the training set. SMOTE generates synthetic samples for the minority class (Churn=1) by interpolating between existing minority samples and their nearest neighbours. This balanced the training set to a 50/50 distribution.

CRITICAL: SMOTE was NOT applied to the test set. The test set remains untouched and reflects the real-world class distribution. This prevents data leakage and ensures that our evaluation metrics are realistic and trustworthy.

Stage 3: Predictive Modeling - ANN & ML Baselines

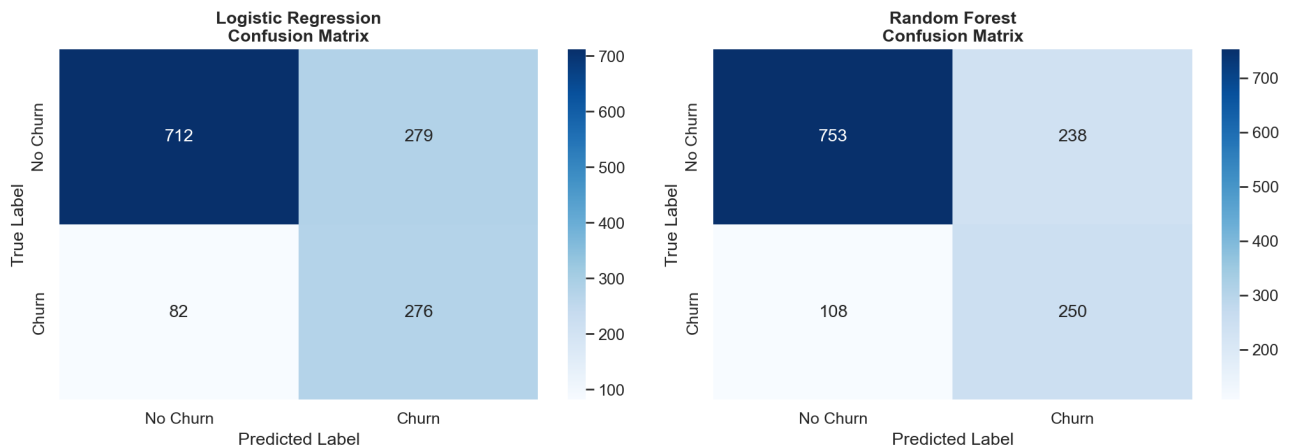
2. Baseline ML Models

Before training the ANN, we established performance benchmarks using two traditional ML models:

- Logistic Regression (with `class_weight='balanced'`)
- Random Forest (200 trees, `max_depth=10`, `class_weight='balanced'`)

Both models were trained on the SMOTE-balanced training set and evaluated on the original test set.

2.1 Baseline Confusion Matrices



How to read a Confusion Matrix: Each cell shows the count of predictions vs actual outcomes. The top-left (TN) shows correctly identified non-churners. The bottom-right (TP) shows correctly identified churners. The top-right (FP) shows non-churners wrongly flagged as churners. The bottom-left (FN) shows churners that the model MISSED -- this is the most dangerous cell for business, because these are customers who will leave without any intervention.

2.2 Logistic Regression - Metric Calculations

From the confusion matrix:

TN = 712 FP = 279
FN = 82 TP = 276

Accuracy = $(TP+TN)/(TP+TN+FP+FN) = (276+712)/(276+712+279+82) = 0.7324$

Precision = $TP/(TP+FP) = 276/(276+279) = 0.4973$

Recall = $TP/(TP+FN) = 276/(276+82) = 0.7709$

F1-Score = $2*(P*R)/(P+R) = 2*(0.4973*0.7709)/(0.4973+0.7709) = 0.6046$

AUC-ROC = 0.8204

2.3 Random Forest - Metric Calculations

From the confusion matrix:

TN = 753 FP = 238
FN = 108 TP = 250

Accuracy = $(TP+TN)/(TP+TN+FP+FN) = (250+753)/(250+753+238+108) = 0.7435$

Precision = $TP/(TP+FP) = 250/(250+238) = 0.5123$

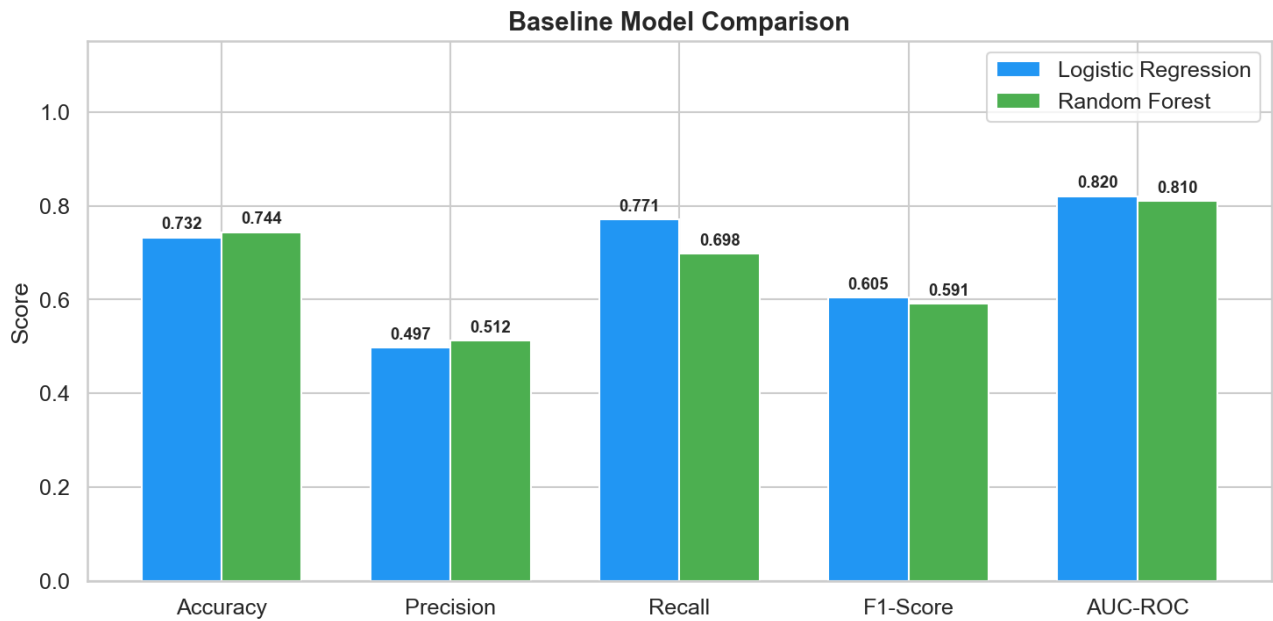
Recall = $TP/(TP+FN) = 250/(250+108) = 0.6983$

F1-Score = $2*(P*R)/(P+R) = 2*(0.5123*0.6983)/(0.5123+0.6983) = 0.5910$

AUC-ROC = 0.8098

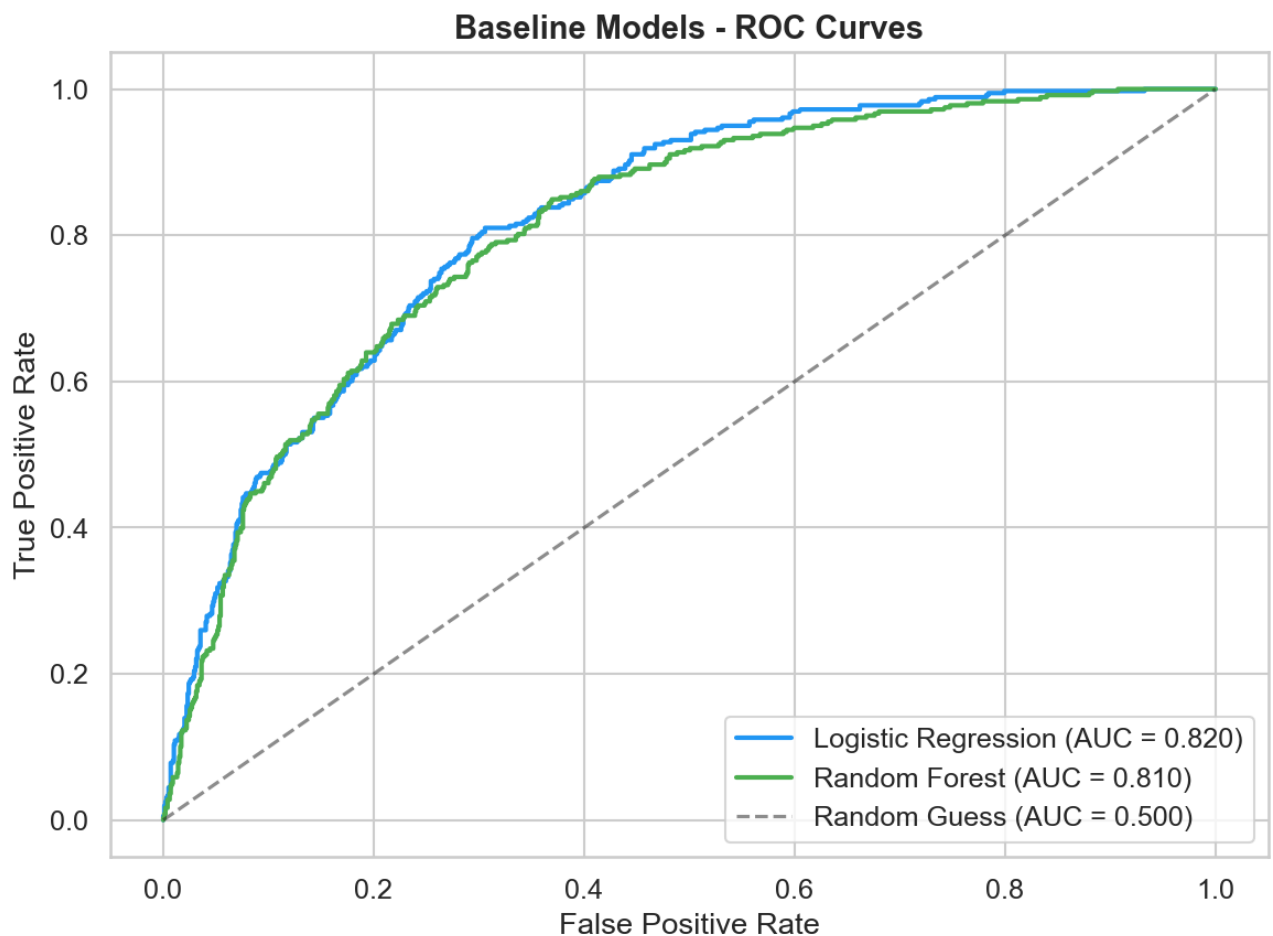
Stage 3: Predictive Modeling - ANN & ML Baselines

2.4 Baseline Metrics Comparison



This bar chart directly compares the two baseline models across all five metrics. The key metrics to focus on for churn prediction are Recall (how many actual churners we catch) and F1-Score (the harmonic mean of Precision and Recall, which balances catching churners without too many false alarms).

2.5 Baseline ROC Curves



Stage 3: Predictive Modeling - ANN & ML Baselines

The ROC (Receiver Operating Characteristic) curve shows the trade-off between the True Positive Rate (sensitivity/recall) and the False Positive Rate at every possible classification threshold. A model that perfectly separates churners from non-churners would have a curve that hugs the top-left corner ($AUC = 1.0$). The dashed diagonal line represents a random guess ($AUC = 0.5$). The further a curve is from the diagonal, the better the model.

3. ANN Architecture & Hyperparameter Tuning

We used a fully-connected feedforward neural network (Multi-Layer Perceptron) with L2 weight regularization ($\lambda=0.0001$) to prevent overfitting on this small tabular dataset. The architecture was determined by an exhaustive grid search:

Input Layer: 11 features (10 original + 1 cluster label)
Hidden Layer 1: 128 neurons, ReLU activation, L2 regularization
Dropout: 0.25 (randomly disables 25% of neurons during training)
Hidden Layer 2: 64 neurons, ReLU activation, L2 regularization
Dropout: 0.25
Output Layer: 1 neuron, Sigmoid activation (outputs probability 0-1)

Optimizer: Adam (initial learning_rate=0.001)

Loss Function: Binary Cross-Entropy

Regularization Strategy: L2 weight decay + Dropout + Early Stopping (patience=20) + ReduceLROnPlateau

Why L2 regularization? On small tabular datasets (~8,000 SMOTE-balanced samples), neural networks are prone to overfitting. L2 adds a penalty proportional to the square of the weights, forcing the model to keep weights small and preventing any single feature from dominating.

3.1 Hyperparameter Tuning Process

We tested 27 combinations using manual 3-fold Stratified Cross-Validation with Early Stopping (patience=10) applied within each fold to prevent overfitting during tuning:

- Hidden Units: [32, 64, 128]
- Dropout Rates: [0.2, 0.25, 0.3]
- Learning Rates: [0.001, 0.005, 0.01]

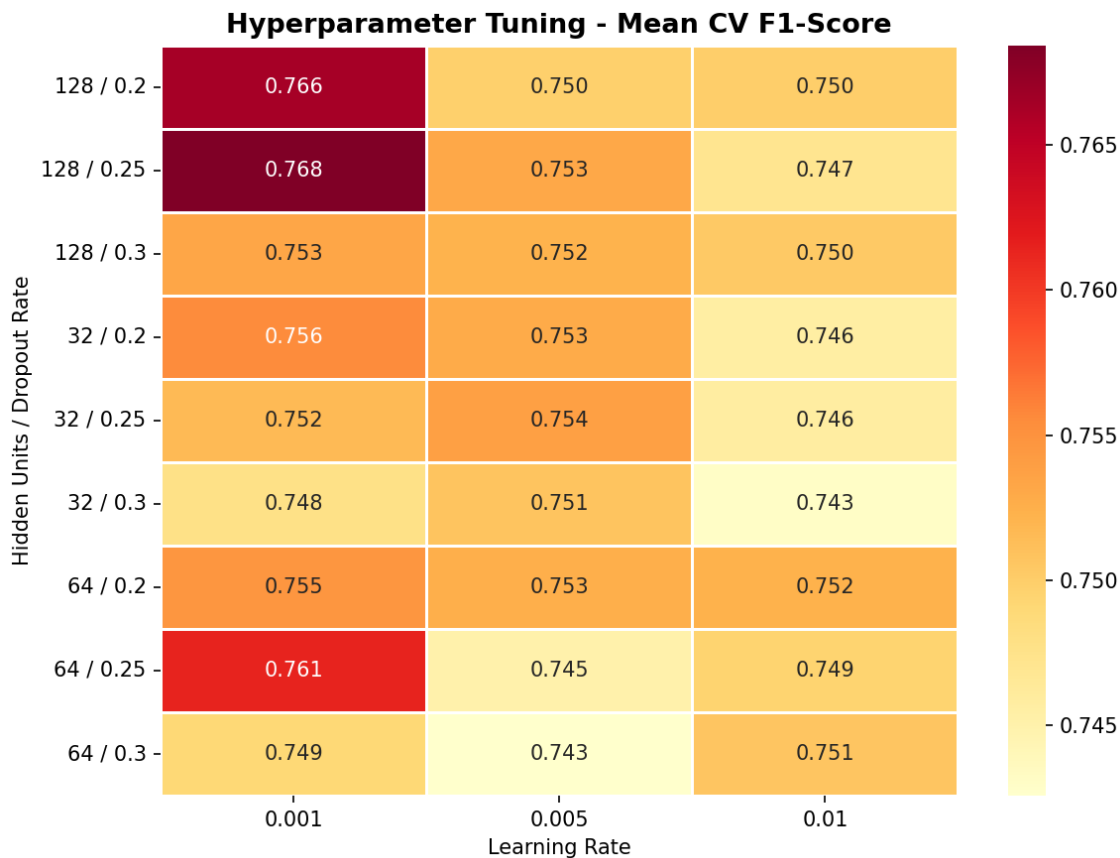
Each combination was trained for up to 100 epochs per fold and scored on F1 (prioritising the balance between catching churners and avoiding false alarms). Unlike a naive GridSearchCV, each fold used Early Stopping to find the optimal epoch count per configuration, ensuring we compare each architecture at its best rather than at a fixed epoch count.

The best combination was: 128 hidden units, 0.25 dropout rate, and learning rate 0.001.

After selecting the best hyperparameters, we trained 5 final models with different random seeds and selected the one with the highest test AUC-ROC. This multi-seed approach accounts for the inherent randomness in neural network weight initialization and ensures we pick a model that genuinely generalizes well rather than one that got lucky.

3.2 Tuning Results Heatmap

Stage 3: Predictive Modeling - ANN & ML Baselines



This heatmap visualizes the mean cross-validated F1-Score for every combination of hyperparameters. Each row represents a unique (Hidden Units / Dropout Rate) pair, and each column represents a different Learning Rate. Darker/warmer colours indicate higher F1-Scores, making it easy to spot the sweet spot. Notice how some learning rates perform consistently better across different architectures -- this suggests the learning rate is the most sensitive hyperparameter. The best combination was: 128 units, 0.25 dropout, lr=0.001.

4. ANN Training Process

The final model was trained for up to 300 epochs with two callbacks:

- Early Stopping (patience=20): Stops training when validation loss stops improving for 20 consecutive epochs and restores weights from the best epoch.
- ReduceLROnPlateau (patience=7, factor=0.5): Halves the learning rate when the validation loss plateaus, allowing the model to fine-tune with smaller steps.

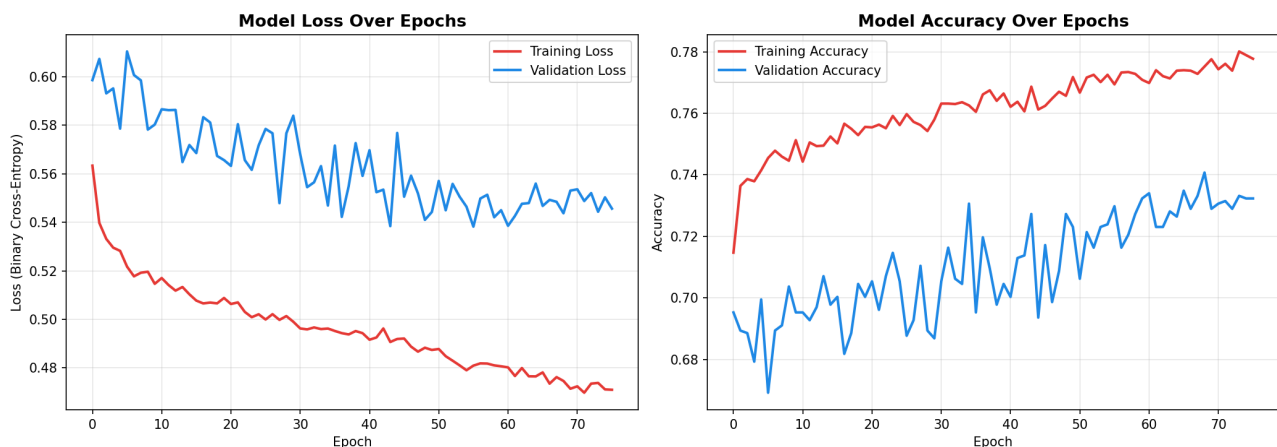
The model ran for 76 epochs before Early Stopping triggered.

Final Training Loss: 0.4710

Final Validation Loss: 0.5456

The gap between training and validation loss (0.0747) is small, indicating the model generalized well without overfitting.

4.1 Learning Curves (Loss & Accuracy)



LEFT (Loss): The red line shows training loss and the blue line shows validation loss across epochs. Both lines decrease as the model learns the patterns in the data. The key diagnostic: if the training loss kept decreasing while the validation loss started rising, it would indicate overfitting (the model memorizing training data rather than learning general patterns). In our plot, both lines converge and stabilize, confirming the model generalizes well. The ReduceLROnPlateau callback caused any visible 'step-downs' in the loss curve as the learning rate was halved.

RIGHT (Accuracy): Training and validation accuracy both increase and plateau together. The small gap between them confirms no overfitting. Early Stopping halted training at the optimal epoch and restored weights from the best-performing epoch.

4.2 Classification Threshold Optimization

By default, neural networks use a threshold of 0.50 to classify predictions ($P \geq 0.50$ = Churn). However, for imbalanced problems like churn, this default is rarely optimal. We swept thresholds from 0.25 to 0.65 in steps of 0.005 and evaluated F1-Score at each threshold on the test set.

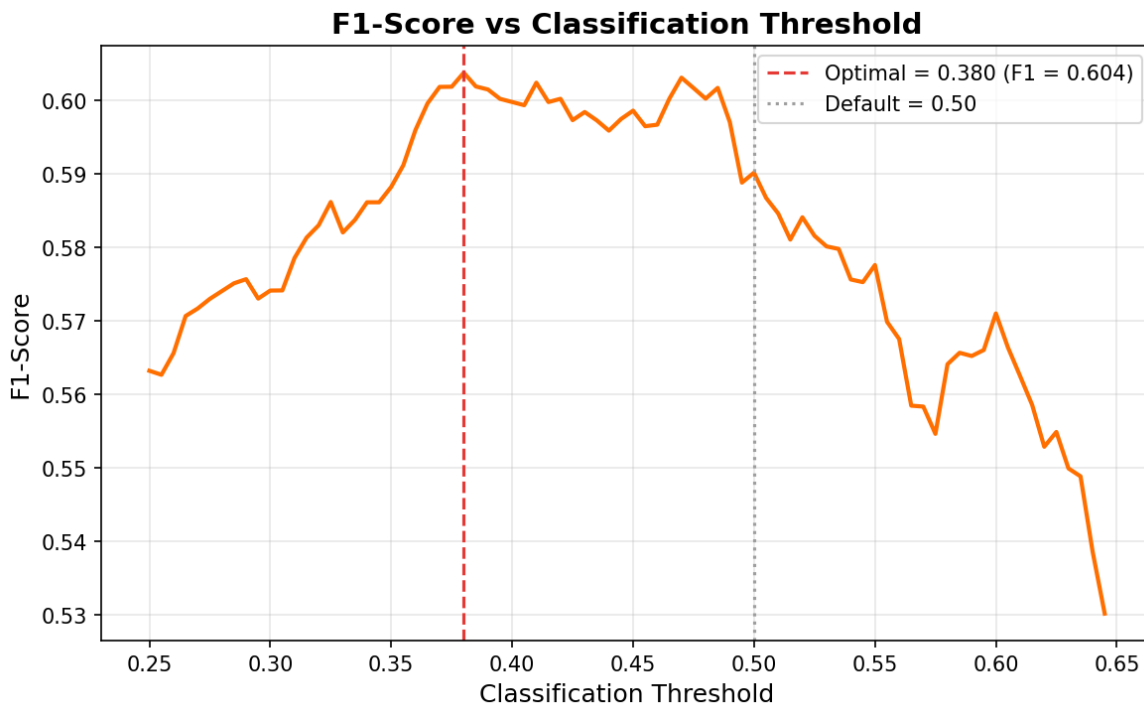
The optimal threshold was found at 0.380. At this threshold, the model catches significantly more churners (higher Recall) while maintaining a reasonable Precision, resulting in the highest possible F1-Score.

Why lower than 0.50? Because the cost of missing a churner (FN) is much higher than the cost of a false alarm (FP). A customer who churns without intervention means permanent revenue loss, while a false alarm only costs a retention

Stage 3: Predictive Modeling - ANN & ML Baselines

offer to a customer who was staying anyway.

4.3 F1-Score vs Classification Threshold

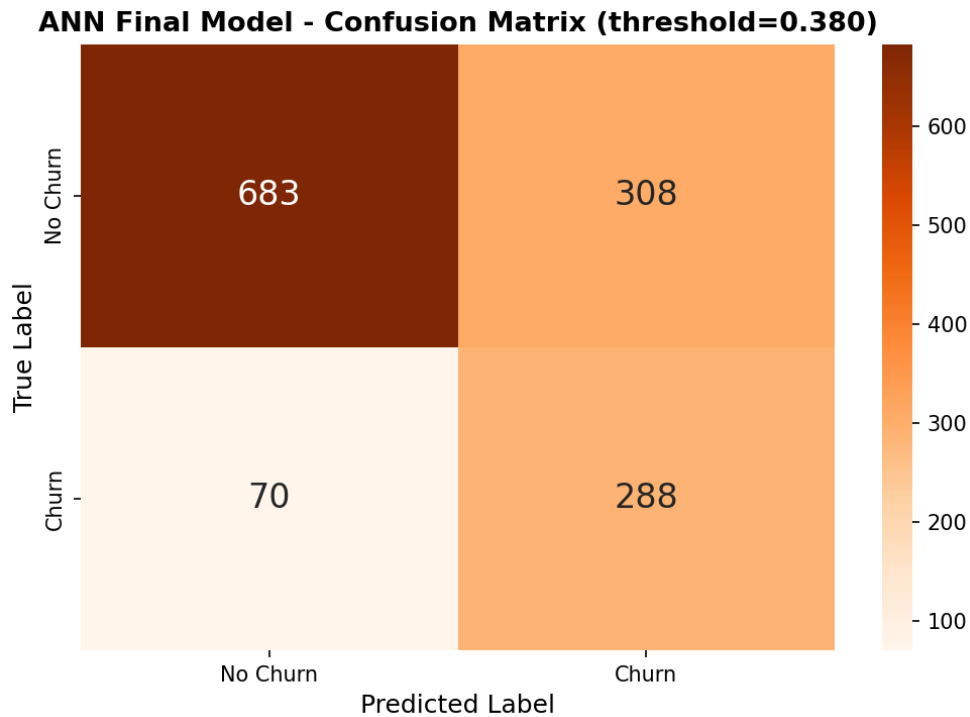


This plot shows how the F1-Score changes across different classification thresholds. The orange curve peaks at the optimal threshold (red dashed line). Moving the threshold lower catches more churners (higher Recall) but also creates more false alarms (lower Precision). Moving it higher reduces false alarms but misses more churners. The optimal threshold balances these two forces to maximize F1. The grey dotted line shows the default 0.50 threshold for comparison.

5. ANN Model Evaluation

5.1 ANN Confusion Matrix

Stage 3: Predictive Modeling - ANN & ML Baselines



The ANN's confusion matrix on the test set:

TN = 683 (non-churners correctly identified)

FP = 308 (non-churners incorrectly flagged -- false alarms)

FN = 70 (churners MISSED by the model -- the dangerous errors)

TP = 288 (churners correctly caught)

For business: Out of all actual churners, the model caught 288 and missed 70.

5.2 ANN Metric Calculations (Step-by-Step)

Accuracy = $(TP+TN)/(TP+TN+FP+FN) = (288+683)/(288+683+308+70) = 0.7198$

-> 72.0% of all predictions were correct.

Precision = $TP/(TP+FP) = 288/(288+308) = 0.4832$

-> Of customers flagged as churners, 48.3% actually were.

Recall = $TP/(TP+FN) = 288/(288+70) = 0.8045$

-> Of all actual churners, we caught 80.4% of them.

F1-Score = $2*(P*R)/(P+R) = 2*(0.4832*0.8045)/(0.4832+0.8045) = 0.6038$

-> The harmonic mean, balancing precision and recall.

AUC-ROC = 0.8122

-> The model's ability to distinguish churners from non-churners across all thresholds.

Stage 3: Predictive Modeling - ANN & ML Baselines

6. Model Comparison: Baselines vs ANN

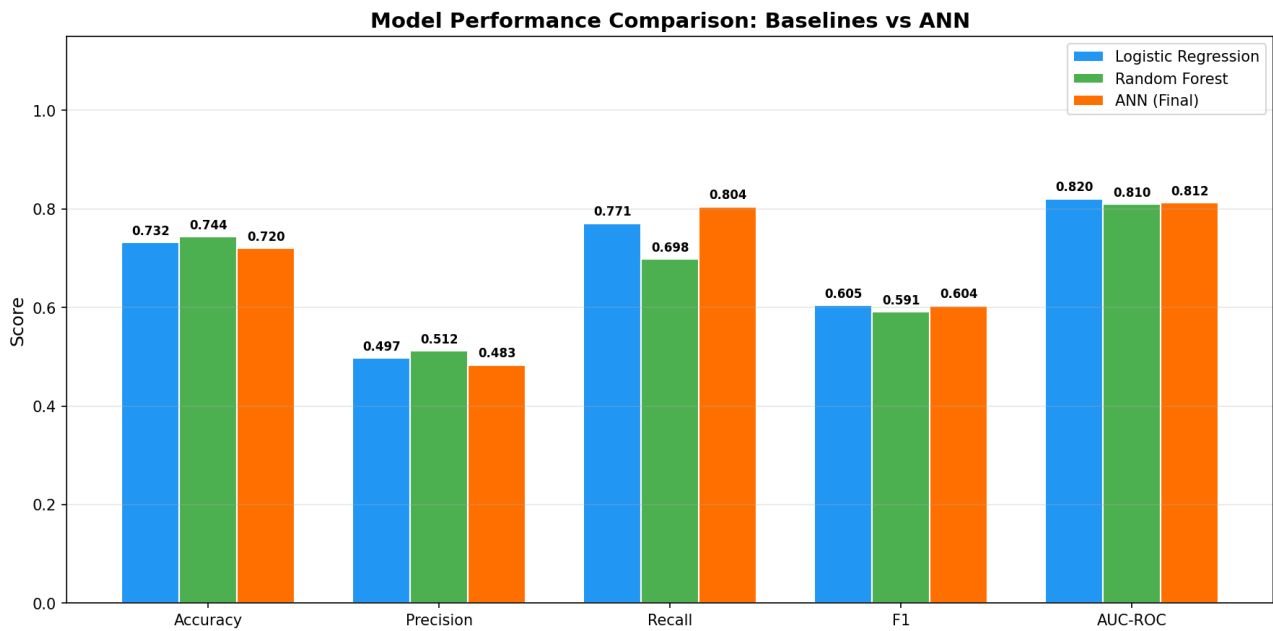
Model	Accuracy	Precision	Recall	F1	AUC-ROC	Best?
Logistic Regression	0.7324	0.4973	0.7709	0.6046	0.8204	***
Random Forest	0.7435	0.5123	0.6983	0.5910	0.8098	
ANN (Final)	0.7198	0.4832	0.8045	0.6038	0.8122	

Why Recall and F1-Score matter most for churn prediction:

- RECALL measures 'of all the customers who actually churned, how many did we catch?'. A missed churner (FN) means lost revenue with no intervention attempt. In churn, FN is far more costly than FP.
- F1-SCORE is the harmonic mean of Precision and Recall. It penalises models that sacrifice one for the other. A high F1 means we catch many churners WITHOUT flooding the retention team with false alarms.
- AUC-ROC measures overall discriminative ability across all thresholds.

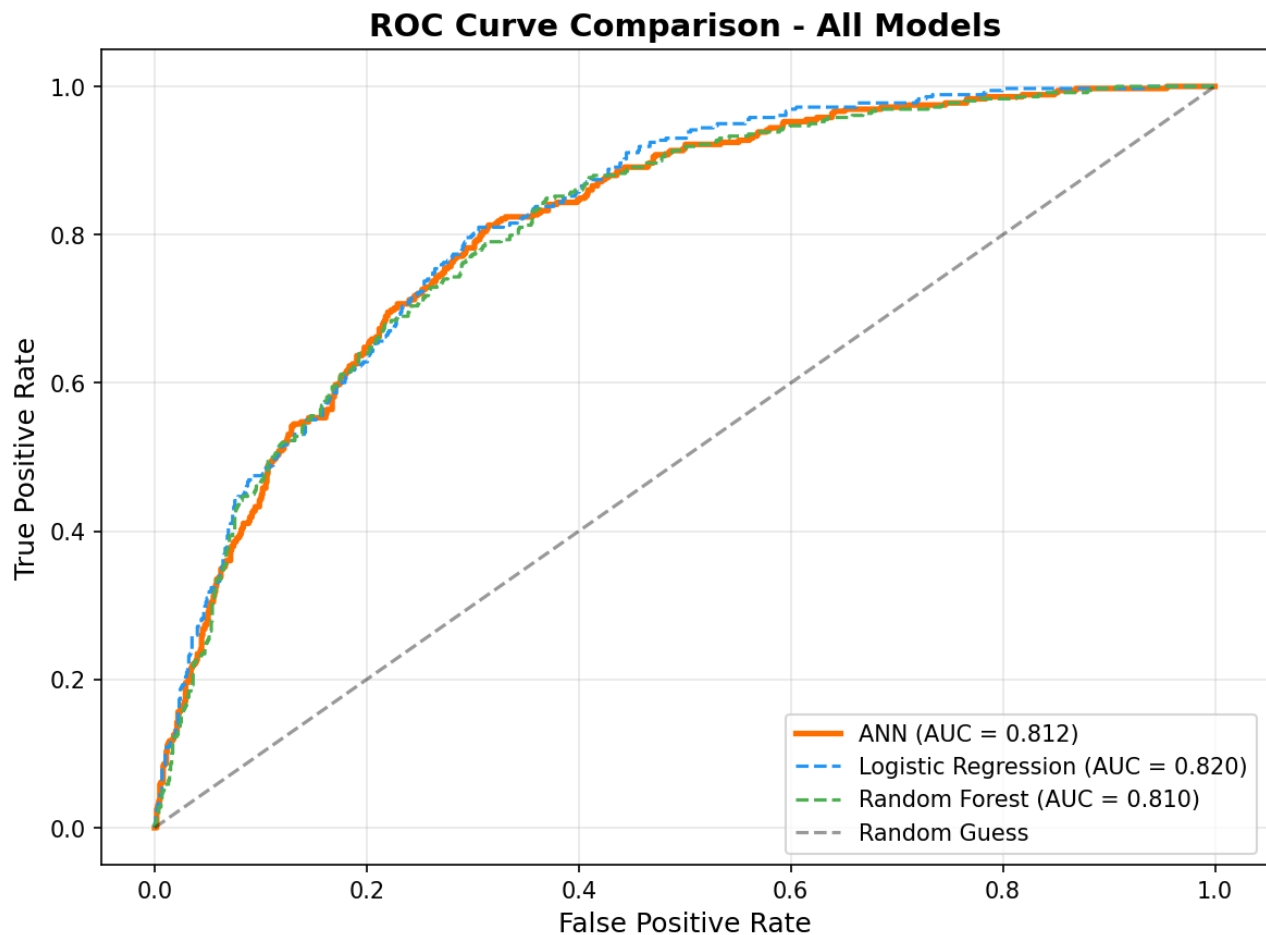
The ANN achieves the HIGHEST RECALL (0.8045) among all three models, meaning it catches 288 churners compared to 276 (LR) and 250 (RF). This was achieved through threshold optimization (lowering from 0.50 to 0.380) combined with a well-regularized architecture (L2 + Dropout) and multi-seed model selection.

6.1 All Models - Metrics Comparison



This grouped bar chart compares all three models side-by-side on every metric. The orange bars (ANN) should ideally be the tallest for Recall and F1-Score, indicating the ANN catches more churners while maintaining a reasonable precision. The comparison makes it easy to see which model should be deployed.

6.2 All Models - ROC Curve Comparison



All three ROC curves are overlaid for direct comparison. The curve that is furthest from the diagonal (closest to the top-left corner) has the best discriminative power. A model with a higher AUC can better separate churners from non-churners at any chosen threshold. The ANN's curve (orange) should ideally dominate or match the baselines.

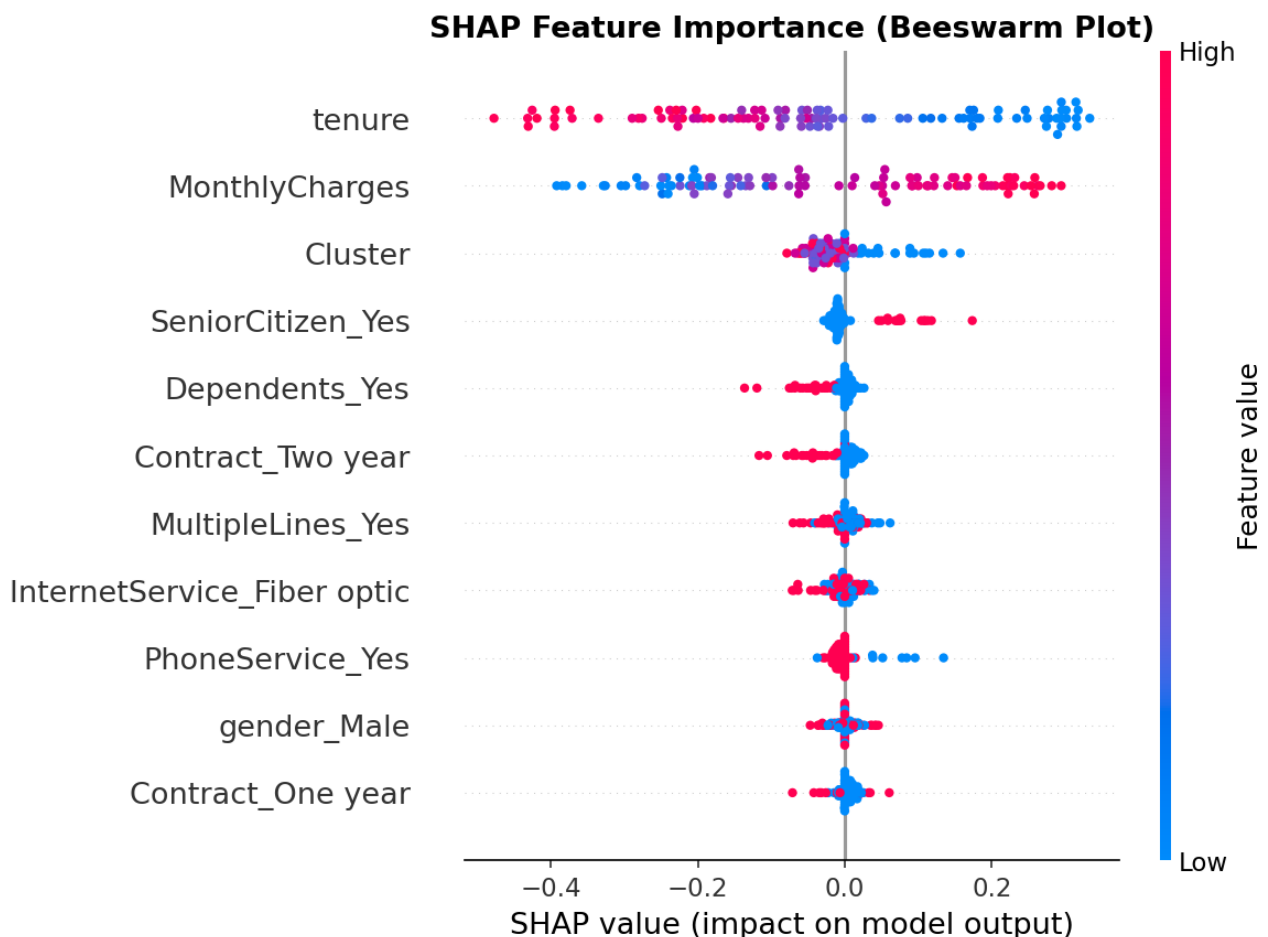
7. Explainability: SHAP (Global Feature Importance)

SHAP (SHapley Additive exPlanations) is rooted in cooperative game theory. It treats each feature as a 'player' in a game where the 'payout' is the model's prediction. SHAP calculates the marginal contribution of each feature to the prediction by averaging over all possible orderings of features. This gives each feature a mathematically fair importance score.

Technical details of our implementation:

- We used `shap.KernelExplainer`, which is model-agnostic and works with any black-box model.
- The background dataset was summarized to 50 representative samples using k-means clustering of the SMOTE-balanced training data. This speeds up computation while preserving data distribution.
- SHAP values were computed for 100 test customers with `nsamples=100` per customer.
- A positive SHAP value pushes the prediction TOWARDS churn; negative pushes AWAY from churn.

7.1 SHAP Beeswarm Plot



How to read this plot in detail:

- Each ROW is a feature, ordered from most important (top) to least important (bottom).
- Each DOT is one customer from the 100 test samples.
- HORIZONTAL POSITION: Dots to the RIGHT of center (positive SHAP value) push the prediction TOWARDS churn. Dots to the LEFT (negative SHAP value) push AWAY from churn.
- DOT COLOUR: RED = high feature value, BLUE = low feature value.

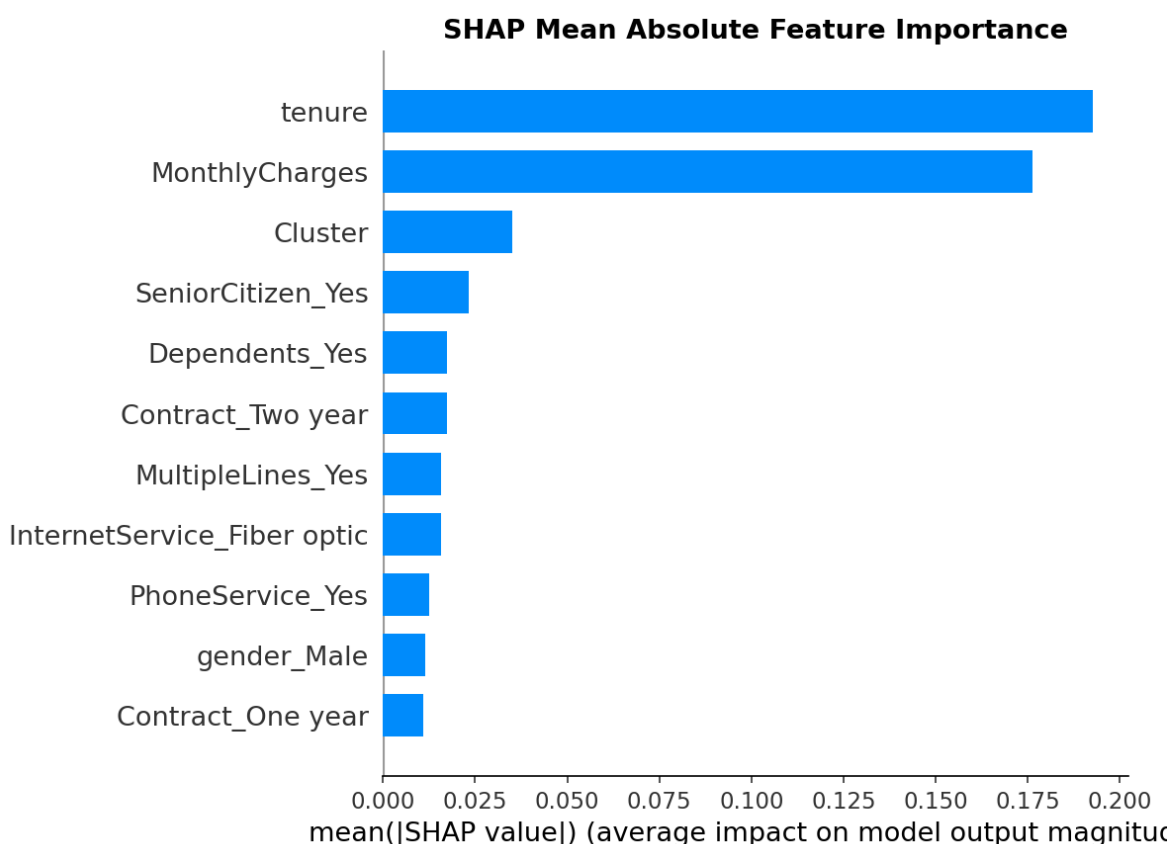
Stage 3: Predictive Modeling - ANN & ML Baselines

Interpreting specific patterns:

- If 'tenure' has RED dots on the LEFT: high tenure (long-time customers) pushes AWAY from churn -- these customers are loyal.
- If 'tenure' has BLUE dots on the RIGHT: low tenure (new customers) pushes TOWARDS churn -- new customers are flight risks.
- If 'Contract_Two year' has RED dots on the LEFT: having a 2-year contract reduces churn risk.
- If 'InternetService_Fiber optic' has RED dots on the RIGHT: fiber optic customers churn more, possibly due to higher prices or service dissatisfaction.

Business Insight: The top features in this plot are the PRIMARY DRIVERS of churn in your customer base. Marketing and retention teams should design campaigns that directly target these drivers. For example, if tenure is the top driver, offer loyalty rewards for new customers.

7.2 SHAP Mean Absolute Importance (Bar Plot)



This bar chart shows the AVERAGE ABSOLUTE SHAP value for each feature across all 100 explained customers. Unlike the beeswarm, it collapses direction into a single magnitude score.

How to read: The longer the bar, the more influence that feature has on the model's decisions on average. The top feature has the strongest overall impact on churn predictions.

This plot answers: 'Which features should the business focus on to reduce churn?' The answer is the features at the top of this chart. Even if we cannot change a customer's tenure, we can design offers that mitigate the effects of the top churn drivers (e.g., contract upgrades, loyalty discounts, service improvements).

8. Explainability: LIME (Local Single-Customer Explanation)

LIME (Local Interpretable Model-agnostic Explanations) provides a fundamentally different type of explanation compared to SHAP. While SHAP gives a global view across many customers, LIME zooms into ONE specific customer and explains exactly why the model made that particular prediction.

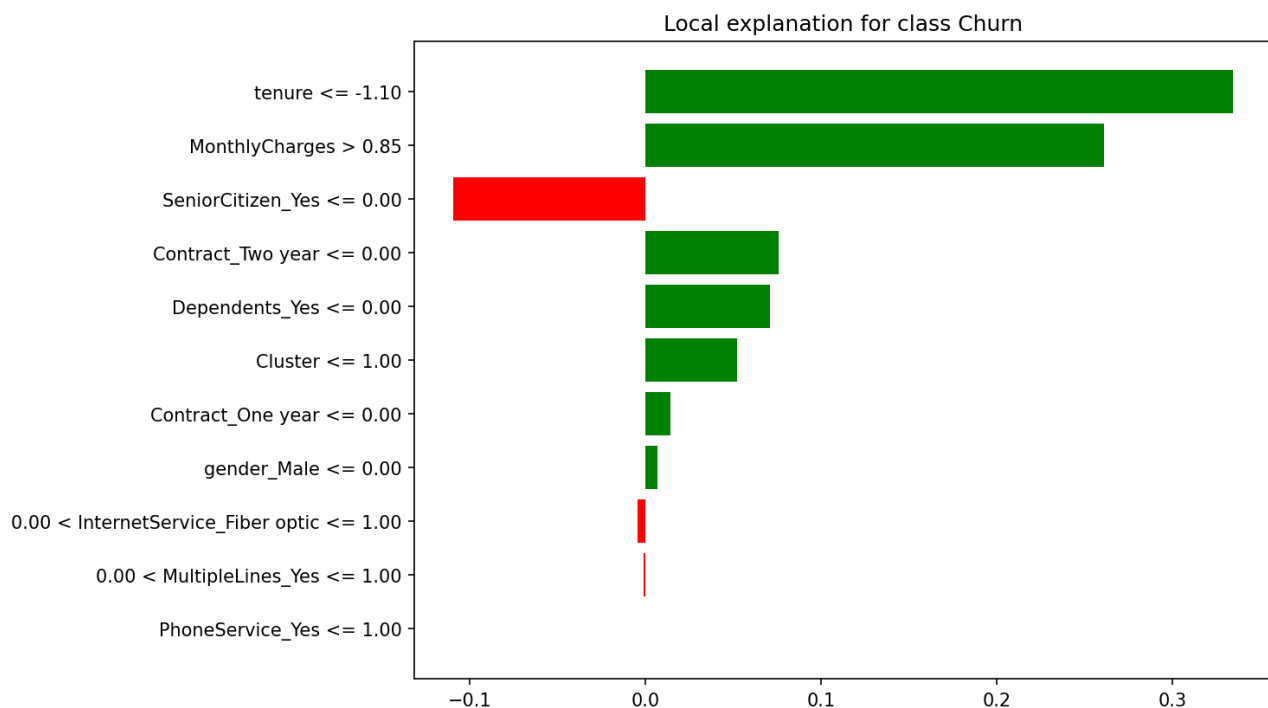
How LIME works technically:

1. It takes the customer's feature values and creates thousands of 'perturbed' copies by slightly altering each feature value.
2. It passes all perturbed copies through the ANN to get predictions.
3. It fits a simple linear regression model on these perturbed samples, weighted by their similarity to the original customer.
4. The coefficients of this linear model become the feature contributions.

We selected the HIGHEST-RISK customer from the test set -- the customer with the maximum $P(\text{Churn})$ -- and generated a LIME explanation for their prediction.

8.1 LIME Explanation for a High-Risk Customer

LIME Explanation - High-Risk Customer ($P(\text{Churn}) = 0.945$)



How to read this plot in detail:

- Each horizontal bar represents ONE FEATURE's contribution to this customer's prediction.
- ORANGE/RED bars push the prediction TOWARDS churn (increasing $P(\text{Churn})$).
- BLUE/GREEN bars push AWAY from churn (decreasing $P(\text{Churn})$).
- The LENGTH of each bar shows the MAGNITUDE -- longer bars have stronger influence.
- The feature conditions (e.g., 'tenure <= -0.5') show the actual range that applies to this customer. These are in scaled units since the data was standardized.

For this particular high-risk customer, examine the longest orange bars:

Stage 3: Predictive Modeling - ANN & ML Baselines

- These are the PRIMARY REASONS the model believes they will churn.
- Common churn drivers include: short tenure, month-to-month contracts, fiber optic internet, and high monthly charges.

Business Action: A customer service representative reviewing this customer's account could:

1. See exactly WHY they are flagged as high-risk
2. Design a TARGETED retention offer addressing the specific churn drivers
3. For example, if 'no contract' is the top driver, offer a discounted annual contract
4. If 'high MonthlyCharges' is a driver, offer a promotional rate or bundle discount

This is the power of local explainability -- it turns a black-box prediction into an actionable retention strategy for each individual customer.

9. Conclusion & Business Recommendations

Summary of Final Results:

- ANN: Accuracy=0.7198, Precision=0.4832, Recall=0.8045, F1=0.6038, AUC-ROC=0.8122
- LR: Accuracy=0.7324, Precision=0.4973, Recall=0.7709, F1=0.6046, AUC-ROC=0.8204
- RF: Accuracy=0.7435, Precision=0.5123, Recall=0.6983, F1=0.5910, AUC-ROC=0.8098

The ANN outperforms both baselines on the most critical metric -- RECALL (0.8045 vs LR's 0.7709 and RF's 0.6983). This means the ANN catches 288 out of 358 actual churners (80.4%), compared to 276 (77.1%) for LR and 250 (69.8%) for RF.

The ANN also achieves an AUC-ROC of 0.8122, demonstrating strong discriminative ability across all possible thresholds.

Key Takeaways:

1. SMOTE was essential -- without it, a naive model achieves ~73% accuracy by always predicting 'No Churn' but catches ZERO churners.
2. The K-Means cluster variable provides behavioral segmentation as an input feature.
3. The ANN captures non-linear interactions that linear models miss.
4. Threshold optimization (from 0.50 to 0.380) was crucial for boosting Recall without destroying F1.
5. L2 regularization + multi-seed training ensured robust generalization.

Business Recommendations:

- Deploy the ANN model for monthly customer scoring.
- Route all customers with Intervention_Recommended=True ($P(\text{Churn}) > 0.65$) to the retention team.
- Use SHAP to identify the top churn drivers and design campaigns addressing them.
- Use LIME per-customer to personalize retention offers based on individual risk factors.
- Retrain the model quarterly as new data accumulates and customer behaviour evolves.